@CACHEPLANE/CHAT · ENTERPRISE GUIDE

The Enterprise Guide to Agent Chat Interfaces in Angular

Production agent chat UI in days, not sprints

cacheplane.io · 2026

Contents

01 The Sprint Tax

02 Batteries-Included Components

03 Theming & Design System Integration

04 Generative UI in Chat

05 Debug Tooling

CHAPTER 1

The Sprint Tax

Chapter 3: The Sprint Tax

Every Angular team building an agent application eventually arrives at the same moment: the backend is ready. The LLM is wired up, the tools are registered, the streaming endpoint is live. Then someone opens a new component file and types `chat.component.ts`, and the next six weeks disappear.

This is the sprint tax—the recurring, predictable cost of building chat UI infrastructure that every team pays independently, from scratch, as though it hasn't been solved before.

The Inventory

A production-ready agent chat interface is not a text box and a message list. The actual surface area looks like this:

- **Message rendering**: Markdown parsing, code block syntax highlighting, sanitized HTML output, role-based layout differentiation
- **Streaming token display**: Character-by-character or chunk-based rendering, cursor animation, partial message state, graceful handling of dropped connections and mid-stream errors
- **Tool call cards**: Visual representation of tool invocations, argument display, result rendering, status transitions across pending, executing, succeeded, and failed states
- **Interrupt panels**: Human-in-the-loop checkpoints where the agent pauses and requires user confirmation or structured input before proceeding
- **Auto-scroll**: Scroll-to-bottom on new content, scroll-lock detection when the user navigates up, resume-on-new-message behavior
- **Accessibility**: ARIA live regions for streamed content, keyboard navigation, focus management across dynamic content, screen reader-compatible state announcements
- **Mobile layout**: Keyboard avoidance, viewport height instability on iOS, touch-friendly input affordances, responsive message bubbles

Each item on this list is a contained engineering problem. Together, they constitute four to six weeks of sprint capacity for a senior engineer—before any iteration begins.

Structural Inefficiency, Not a Skill Gap

Teams that spend six weeks building this are not slow. They are absorbing a structural tax that has nothing to do with the differentiated value they are trying to ship. The chat chrome is table stakes. The agent graph, the tool integrations, the domain-specific interrupt logic, the retrieval pipeline—that is the product. The message list is not.

The hidden costs compound the timeline. Accessibility for streaming content is genuinely hard: ARIA live regions can flood screen readers if implemented naively, and managing focus across dynamically inserted content requires deliberate engineering. Streaming display has edge cases that do not appear in demos—what happens when a chunk arrives after the user has already submitted the next message? What does the UI do with a malformed SSE frame? Tool call state machines look simple until you model cancellation, retry, and partial results simultaneously. These problems are not insurmountable, but they take time that no product roadmap budgets honestly.

Demo Versus Production

"Good enough for demo" is a message bubble that renders a full response after a fake typing delay. Production is a component that handles backpressure from a real streaming endpoint, recovers from network interruption without corrupting message state, displays tool execution in real time, and remains navigable by keyboard without visual regression. The gap between those two things is where most teams discover the actual scope of the problem—usually during QA in week five.

The Opportunity Cost

Senior Angular engineers who spend three sprints on chat infrastructure are not spending those sprints on agent integration. They are not building the domain-specific tool interfaces, the custom interrupt flows, or the application state management that ties agent output to the rest of the product. Those are the things that will differentiate the application. The scroll behavior will not.

The Thesis

`@cacheplane/chat` exists to eliminate the sprint tax. Ship the chat UI on day one—message rendering, streaming, tool call cards, interrupt panels, accessibility, mobile layout, all of it—and spend the sprints on what actually matters: the integration work that is specific to your agent, your domain, and your users.

The backend is ready. The chat UI should be too.

CHAPTER 2

Batteries-Included Components

Chapter 4: Batteries-Included Components

Two Tiers, One Decision

`@cacheplane/chat` ships two distinct component tiers. The prebuilt tier gives you a production-ready chat interface with a single element. The headless tier gives you behavior and state without a single line of CSS opinion. Your team picks the tier that matches how much control you actually need — not how much you think you might need someday.

Most teams start with prebuilt and never leave. That is the correct call.

The Headless Tier

Headless components own the hard parts: state management, message streaming, keyboard interactions, accessibility semantics, and agent lifecycle integration. They render nothing styled.

The four primitives are:

- **`chat-messages`** — subscribes to the agent's `AIMessage[]` signal and handles streaming updates, scroll anchoring, and message-level change detection
- **`chat-input`** — manages the textarea lifecycle, submission gating (blocking sends mid-stream), and keyboard shortcuts
- **`chat-tool-calls`** — surfaces in-flight and completed tool invocations with slot outlets for custom rendering
- **`chat-interrupt`** — exposes abort controls tied directly to the agent's cancellation token

Each primitive accepts an `agentRef` input and wires itself to the shared agent context. You bring the markup and styles. The component brings everything else. This is the right choice when your design system has strict structural requirements or when you are embedding agent interactions inside an existing, highly composed UI.

The Prebuilt Tier

`chat-prebuilt` is a single component that composes all four headless primitives internally, applies a production-quality default theme, and handles responsive layout. Zero configuration is required beyond connecting an agent reference.

```
import { AgentService } from '@cacheplane/angular';
import { ChatPrebuiltComponent } from '@cacheplane/chat';
@Component({
  selector: 'app-support',
  imports: [ChatPrebuiltComponent],
  template: `


  `
})
export class SupportComponent {
  constructor(public agentService: AgentService) {}
}
```

That is a fully functional streaming chat interface. Message history, input handling, tool call display, and interrupt controls are all included.

How the Component Model Connects

Both tiers consume the `AgentRef` produced by `@cacheplane/angular`. The `AgentRef` is a stable reference that exposes an `AIMessage[]` signal, a send method, and a cancellation interface. Components bind to this ref and react to signal emissions via Angular's standard reactivity primitives — no custom change detection strategies required, no zone workarounds.

When the agent emits a new token mid-stream, the `AIMessage[]` signal updates, and `chat-messages` incrementally patches the DOM. The mechanism is straightforward signal subscription. There is no proprietary diffing layer to reason about.

Composing the Two Tiers

The tiers are not mutually exclusive. A common pattern: use `chat-prebuilt` for the standard chat surface, then replace specific sections with headless primitives where your requirements diverge. If your design requires a custom tool call renderer — perhaps

displaying structured data in a proprietary grid — you drop `chat-tool-calls` into that slot directly and style it yourself while leaving the rest of `chat-prebuilt` intact.

When to Migrate

Start with `chat-prebuilt`. Migrate a section to headless when a specific component's default rendering is structurally incompatible with your design system — not when you want to tweak padding or colors, which CSS custom properties handle without touching the component tier at all.

Headless is not a reward for complexity. It is a tool for genuine structural divergence.

CHAPTER 3

Theming & Design System Integration

Chapter 6: Theming & Design System Integration

A chat interface that looks like it came from a SaaS starter kit is a trust problem. Users notice when a modal, sidebar, or embedded panel breaks the visual contract established by the rest of your product. For enterprise teams, that inconsistency signals a lack of ownership — and in agent-facing tools, ownership matters. This chapter covers how to bring `@cacheplane/chat` fully into your design system without forking component source or writing brittle CSS overrides.

The CSS Custom Property API

`@cacheplane/chat` exposes all visual decisions as CSS custom properties scoped under the `--cp-` namespace. These are not internal implementation details — they are the public API for visual configuration. Properties are declared on the host element and cascade normally through Shadow DOM boundaries where applicable, giving you a single override surface regardless of where components are mounted in the Angular component tree.

The token surface covers six categories: color (surface, text, border, accent, semantic), typography (family, size scale, weight, line height), spacing (component padding, message gap, input height), shape (border radius at three scale points), motion (transition duration and easing), and elevation (box shadow values).

Design Token Mapping

If your design system already emits CSS custom properties — through Style Dictionary, Theo, or a Figma Tokens pipeline — mapping to `@cacheplane/chat` is a one-to-one alias operation. Define the mappings in a single stylesheet or Angular style encapsulation block. Avoid hardcoding raw values into `--cp-` properties directly; alias through your own tokens so that upstream design system changes propagate automatically.

Typography Integration

Font family, size scale, and line height are independently configurable. `--cp-font-family` accepts any valid CSS font stack. Size tokens (`--cp-text-sm`, `--cp-text-base`, `--cp-text-lg`) map to the message body, metadata labels, and heading contexts respectively. If your type scale uses `rem` values derived from a root size override, `@cacheplane/chat` respects document root sizing — no unit conversion required.

Color System Integration

Surface tokens (`--cp-surface-base`, `--cp-surface-raised`, `--cp-surface-input`) control background layering. Text tokens handle primary content, secondary metadata, and disabled states. `--cp-accent` drives interactive elements — send buttons, link text, focus rings. Semantic tokens (`--cp-color-error`, `--cp-color-warning`, `--cp-color-success`) control inline status indicators on message delivery states and tool call results. Map these directly to your semantic palette.

Dark Mode

Because the entire visual surface is token-driven, dark mode requires no component changes. Apply alternate token values under a `[data-theme="dark"]` attribute selector or a `prefers-color-scheme` media query. The component renders whatever the cascade resolves. If your Angular application manages theme switching through a class or attribute on `` or ``, the chat components respond automatically.

Brand Override Example

```
cp-chat-window {
  --cp-font-family: 'Inter', sans-serif;
  --cp-surface-base: var(--ds-background-primary);
  --cp-surface-raised: var(--ds-background-elevated);
  --cp-surface-input: var(--ds-background-secondary);
  --cp-text-primary: var(--ds-text-default);
  --cp-text-secondary: var(--ds-text-subtle);
  --cp-accent: var(--ds-color-brand-500);
  --cp-border-radius-md: var(--ds-radius-md);
  --cp-color-error: var(--ds-color-critical);
  --cp-transition-duration: 150ms;
}
```

Ten properties. Full brand alignment. No component source required.

Token Limits and the Headless Tier

CSS custom properties cannot retheme structural decisions: message bubble layout, avatar placement geometry, the composition of the input toolbar, or the ordering of action elements. If your design system requires layout-level divergence — for example, a top-mounted input bar or a two-column transcript layout — the token API will not reach that far. That is the correct boundary for dropping to the headless tier, where `@cacheplane/chat/headless` exposes behavior and state without any rendered output, and your team owns the template entirely. Use tokens for brand alignment; use headless for structural ownership.

CHAPTER 4

Generative UI in Chat

Chapter 7: Generative UI in Chat

Text responses have a ceiling. When a financial agent needs to surface a sortable data table or a scheduling agent needs to render an interactive booking form, prose falls short. Generative UI closes that gap by allowing agents to emit structured UI specifications directly into the message stream — specifications that the chat interface resolves into live Angular components.

How Generative UI Messages Work in the Stream

From the chat renderer's perspective, a generative UI message is just another message type. The message stream carries a `type` discriminator — `text`, `tool_call`, `ui_spec`, and so on. When `@cacheplane/chat` encounters a `ui_spec` message, it routes it to the UI renderer instead of the text pipeline. The result is that a data table, approval card, or booking form appears inline between conversational turns, indistinguishable in position from any other message bubble, but rendered as a fully interactive Angular component.

The json-render Spec

The json-render specification is a JSON-based declarative format for describing UI trees. An agent emits a payload describing component type, props, and children. The chat layer deserializes that payload and resolves each component node against a registry. Because the spec is declarative and serializable, it travels cleanly over SSE or WebSocket streams and requires no client-side code changes when an agent starts emitting new component types — only a registry entry.

Google's A2UI Spec

A2UI extends the base idea with agent-specific primitives that json-render doesn't cover. It introduces first-class concepts for approval flows (a structured action requires explicit user confirmation before the agent proceeds), structured data cards with typed field schemas, and multi-step task UI. Where json-render is a general rendering protocol, A2UI is opinionated about agent interaction patterns. `@cacheplane/chat` supports both specs out of the box, detecting the discriminator on the incoming message and routing accordingly.

Integration with @cacheplane/render

The rendering layer is handled by `@cacheplane/render`, which owns component resolution, prop binding, and the registry itself. `@cacheplane/chat` delegates all spec rendering to `@cacheplane/render` — the chat library doesn't need to know how components are resolved, only that the renderer accepts a spec and returns a mounted component. This separation keeps the chat layer focused on message orchestration and keeps the rendering logic reusable outside of chat contexts.

The Registry Pattern in Chat

Custom components are registered once and become available to any agent that knows the component identifier. The registry maps string keys to Angular component classes. Any component the agent might emit — a custom chart, a domain-specific form, a confirmation widget — needs a corresponding entry.

```
import { provideChatRenderer } from '@cacheplane/chat';
import { provideRenderRegistry } from '@cacheplane/render';
import { DataTableComponent } from './components/data-table.component';
import { BookingFormComponent } from './components/booking-form.component';
import { ApprovalCardComponent } from './components/approval-
card.component';
export const appConfig: ApplicationConfig = {
  providers: [
    provideChatRenderer(),
    provideRenderRegistry({
      'data-table': DataTableComponent,
      'booking-form': BookingFormComponent,
      'approval-card': ApprovalCardComponent,
    }),
  ],
};
```

Any component the agent emits by key is resolved here. Unknown keys fall back to a configurable fallback component rather than throwing.

Progressive Rendering via Streaming JSON Patches

Agents don't emit complete UI specs in a single chunk. They stream JSON patches — RFC 6902 operations that incrementally build the spec as the model generates it. `@cacheplane/chat` applies each patch to the in-progress spec and triggers Angular's change detection cycle. The effect is a component that populates live: table rows appear as the agent reasons through the data, form fields materialize in sequence. This isn't a loading spinner followed by a full render — it's the UI itself arriving progressively, which

is both faster to first meaningful paint and more transparent about what the agent is doing.

CHAPTER 5

Debug Tooling

Chapter 7: Debug Tooling

Debugging a running agent is fundamentally different from debugging a REST call. There is no single request to inspect in the network tab. The state lives across a stream of discrete events, tool invocations happen inside the graph before responses surface, and interrupt flows depend on timing between the UI and the server-side checkpoint. Standard browser tooling was not built for this. `chat-debug` was.

What chat-debug Exposes

`chat-debug` is a developer overlay built into `@cacheplane/chat`. When active, it renders a resizable panel alongside your chat interface that gives you a live view of four distinct concerns.

Raw message state surfaces the full `AIMessage[]` array as it exists in the agent signal at any given moment — not a serialized snapshot, but the live signal value. You can watch content chunks arrive and accumulate during streaming without manually logging signal reads.

The streaming event log shows every server-sent event in the order it was received, including event type, timestamp, and raw payload. This is where you catch malformed chunk sequences, unexpected `end` events, and latency gaps between tool execution and the next assistant turn.

The tool call state machine tracks each tool invocation through its full lifecycle: `pending`, `executing`, `complete`, or `error`. For each call you see the tool name, the serialized input payload, the output value, and wall-clock execution time. When a tool is slow or returning unexpected output, this panel removes the guesswork.

Interrupt state shows the full interrupt payload at the moment the graph pauses, including the checkpoint ID and any structured data the graph passed to the UI. After the user responds, the panel records what was submitted. If your interrupt flow has a bug, the before-and-after diff is right there.

Adding chat-debug

Drop `ChatDebugComponent` into any host component that already uses the `@cacheplane/chat` primitives. No providers, no configuration file. In dev mode it

activates automatically.

```
import { ChatDebugComponent } from '@cacheplane/chat/debug';
@Component({
  imports: [ChatDebugComponent],
  template: `



  `
})
export class AppShell {}
```

`chat-debug` reads from the same injected `AgentChatService` instance your chat interface uses. There is no secondary data pipeline and no performance overhead on the message handling path itself.

Inspecting Messages and Tool Calls

Clicking any message in the raw state view expands it into a structured inspector showing message type, role, content blocks, and metadata fields. Streamed messages show their final assembled content alongside a chunk count. Tool call messages link directly to the corresponding entry in the tool call state machine view.

The tool call inspector is the most useful surface for backend integration bugs. Input and output payloads are formatted as syntax-highlighted JSON. Execution timing lets you correlate slow tool responses with latency you see in the event log.

Angular DevTools Integration

Agent signals registered through `@cacheplane/chat` appear as named signals in the Angular DevTools component tree. You can inspect `messageStream`, `interruptState`, and `toolCallRegistry` directly in the DevTools panel without instrumentation code. The signal graph reflects live state, so you can pause, inspect, and resume without affecting the stream.

Production Safety

`chat-debug` uses a build-time token that resolves to a no-op component in production. The overlay never renders, and tree-shaking removes the debug module entirely from the production bundle. No conditional guards in application code are required. The separation is enforced at the library level.

When a stream misbehaves in development, `chat-debug` gives you the full picture in one place. That is its only job, and it does it without adding complexity to your application architecture.